



Universitat
Pompeu Fabra
Barcelona

Reflection Probes

Dídac Hierro Soteras

Marc Bosch Manzano

Gràfics en Temps Real 2025

The Concept

Until this point, we modeled direct lighting with its diffuse and specular components, but we were missing something important: **reflections**.

Materials have some properties that make them more or less reflective.



Cubemaps & Probes

To implement reflections we used **cubemaps**, which render their surroundings as a texture onto each one of their 6 faces, and assign them to the reflective materials of the scene.

What are **reflection probes**? Strategic positions of the scene where to compute the cubemaps.

Why do we need probes? balance efficiency/realism.



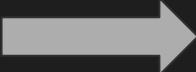
Our Implementation

```
class ReflectionProbeEntity : public SCN::BaseEntity
{
public:
    int width;
    int height;

    vec3 position;
    float range;
    float reflection_strength;

    GFX::Texture cubemap;
    GFX::FBO capture_fbo;

    float sphere_radius;
    GFX::Mesh sphere;
```



```
void ReflectionProbeEntity::captureEnvironment(SCN::Scene* scene, SCN::Renderer* renderer)
{
    const vec3 directions[NUM_FACES] = {
        vec3(1, 0, 0),
        vec3(-1, 0, 0),
        vec3(0, 1, 0),
        vec3(0, -1, 0),
        vec3(0, 0, 1),
        vec3(0, 0, -1)
    };

    const vec3 up_vectors[NUM_FACES] = {
        vec3(0, -1, 0),
        vec3(0, -1, 0),
        vec3(0, 0, 1),
        vec3(0, 0, -1),
        vec3(0, -1, 0),
        vec3(0, -1, 0)
    };

    Camera cam;
    cam.setPerspective(90.0f, 1.0f, 0.1f, 1000.0f);
    cam.eye = root.model.getTranslation();

    for (int face = 0; face < NUM_FACES; face++) {
        cam.lookAt(cam.eye, cam.eye + directions[face], up_vectors[face]);
        capture_fbo.setTexture(&cubemap, face);
        renderer->renderFBO(&cam, &capture_fbo, "forward_light_single_pass");
    }

    glEnable(GL_TEXTURE_CUBE_MAP_SEAMLESS);
    cubemap.generateMipmaps();
}
```

Our Implementation

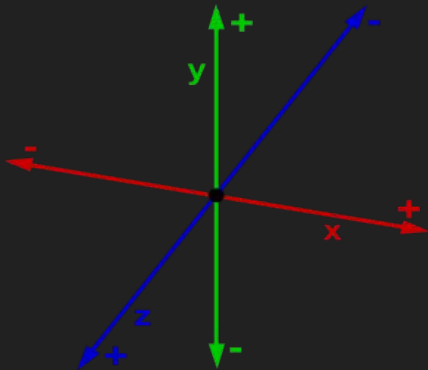
We included the reflections' implementation inside the computation of PBR lighting.

```
vec3 compute_pbr_probes(vec3 light_position, vec3 camera_position,  
                        vec3 world_position, vec3 normal, float roughness, vec3 F0,  
                        vec3 reflection_color, float reflection_strength, float metalness)  
    ...  
  
    vec3 kD = vec3(sqrt(1.0 - metalness));  
    vec3 diffuse_component = N_dot_L * kD;  
  
    ...  
  
    float ref_str = mix(reflection_strength, 1.0, metalness); //(Ref Probes)  
    vec3 reflections = fresnel * reflection_color * ref_str; //(Ref Probes)  
    specular_component += reflections * sqrt((1.0 - roughness)); //(Ref Probes)
```

- More rough $\Rightarrow$ less reflective
- More metallic $\Rightarrow$ more reflective

Our Implementation

Class to create a **grid** of probes.
We can set how many probes
we want to create and set their
position.



```
class ReflectionProbeGrid : public SCN::BaseEntity
```

```
public:
```

```
    vec3 probe_grid_dimensions;  
    float probe_spacing;  
    vec3 probe_grid_origin;  
    std::vector<ReflectionProbeEntity*> reflection_probes;  
    bool render_spheres_mode;
```

```
void ReflectionProbeGrid::create()
```

```
{
```

```
    clear();
```

```
    vec3 grid_size = probe_grid_dimensions * probe_spacing;
```

```
    vec3 half_grid = grid_size * 0.5f;
```

```
    for (int x = 0; x <= probe_grid_dimensions.x; ++x) {
```

```
        for (int y = 0; y <= probe_grid_dimensions.y; ++y) {
```

```
            for (int z = 0; z <= probe_grid_dimensions.z; ++z) {
```

```
                vec3 pos = probe_grid_origin - half_grid + vec3(x, y, z) * probe_spacing;
```

```
                ReflectionProbeEntity* probe = new ReflectionProbeEntity();
```

```
                probe->setPosition(pos);
```

```
                probe->range = probe_spacing * 1.5f;
```

```
                reflection_probes.push_back(probe);
```

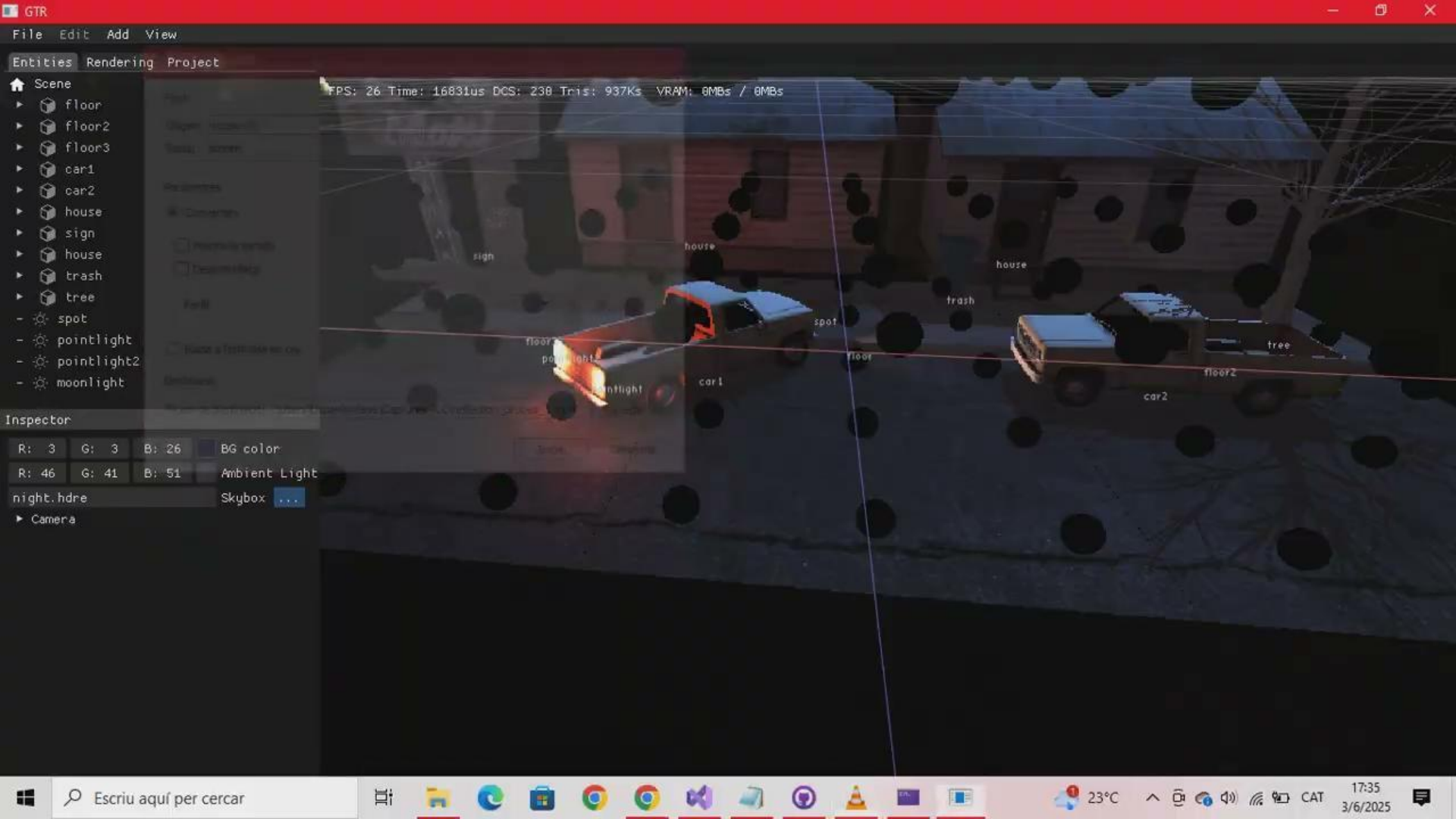
```
            }
```

```
        }
```

```
    }
```

```
}
```

Demo



Challenges

During the process of implementing reflections using probes we encountered some challenges:

- **Memory limitations**
- **GPU computing limitations**

```
OpenGL Error: Memoria insuficiente  
Assertion failed: 0, file C:\Users\Usuari\Documents\GitHub\GTRFrame
```

```
FPS: 6
```

Why? We were rendering each cubemap probe by photogram. Recomputing again and again unnecessarily.

References

[Cubemap Reflections](#)

Thank You!